CO1 – ASSESSMENT TOOL 2

CONSTRAINT-BASED PROBLEM SOLVING

Name	Rishi Varthini S
Registration Number	192421304
Course Code	CSA1735
Course Title	Artificial Intelligence
Faculty Name	Dr. R. Thalapathi Rajasekaran

INTRODUCTION

Constraint-based problem solving models a task using variables, domains, and constraints. A solution assigns a value to each variable while satisfying all constraints. Techniques include backtracking, constraint propagation, MRV, forward checking, and state-space search.

OBJECTIVES

- Represent real-world tasks as constraint satisfaction problems (CSPs).
- Apply backtracking, heuristics, and constraint propagation.
- Identify valid assignments and constraint violations.
- Use search algorithms for route planning and state-space problems.
- Explain intelligent-agent behaviour under uncertainty.

1. HOSPITAL SHIFT SCHEDULING (BACKTRACKING SEARCH)

Problem statement and CSP formulation

Assign three doctors (D1, D2, D3) to Morning, Afternoon, and Night shifts. Variables: D1, D2, D3. Domain of each variable: {Morning, Afternoon, Night}. Constraints:

- Each doctor works exactly one shift.
- D1 must not work Night.
- D2 must work before D3 (Morning < Afternoon < Night).
- Only one doctor may be assigned to each shift.
- D3 cannot work Morning.

Backtracking pseudocode

```
BACKTRACK(assignment):
   if all doctors are assigned:
      return assignment if all constraints hold
   choose an unassigned doctor D
   for each shift in DOMAIN(D):
      if shift is consistent:
         assign D = shift
         result = BACKTRACK(assignment)
         if result is a solution: return result
         remove D from assignment
   return FAILURE
```

Valid assignment and verification

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

This satisfies all constraints: D1 is not on Night, D2 works before D3, every shift has one doctor, and D3 is not on Morning. Backtracking rejects invalid partial schedules before exploring a full assignment.

2. ROBOT NAVIGATION IN A 5 × 5 GRID (HEURISTIC SEARCH)

Problem formulation

A robot moves from Start (S) to Goal (G) in a 5 × 5 grid with obstacles. Legal actions are Up, Down, Left, and Right; every move costs 1.

- State: current grid cell (row, column).
- Initial state: S; goal test: current cell equals G.
- Transition: move to an adjacent cell inside the grid that is not blocked.
- Path cost: number of moves.

Heuristic and A* pseudocode

Use Manhattan distance $h(n)=|\text{row}(n)-\text{row}(G)|+|\text{column}(n)-\text{column}(G)|$. It ignores obstacles and never overestimates the true distance, so it is admissible for four-direction movement.

```
A_STAR(S, G):
   open = priority queue ordered by f(n)=g(n)+h(n)
   g[S] = 0; insert S with priority h(S)
   while open is not empty:
      n = remove node with lowest f
      if n == G: return reconstruct_path(n)
      for each legal neighbour m:
         new_g = g[n] + 1
         if m is new OR new_g < g[m]:
            g[m] = new_g; parent[m] = n
            insert/update m with priority new_g+h(m)
   return FAILURE
```

A* balances cost already travelled, g(n), with estimated remaining cost, h(n). Obstacle checks prevent illegal moves.

3. AUTONOMOUS RESCUE ROBOT (ONLINE SEARCH IN A PARTIALLY OBSERVABLE ENVIRONMENT)

Problem and constraints

A rescue robot explores an unknown disaster-hit building. Sensors reveal only adjacent cells, some paths are blocked, and risky zones add cost. Movement costs 1; entering a risky zone adds a penalty of 2. The robot must update its map while exploring.

Representation and online search

The robot maintains a belief/map state marking cells as known-free, blocked, risky, or unknown. It observes, updates the map, plans a safe low-cost route, moves one step, and replans when new information invalidates the route.

```
ONLINE_RESCUE():
   initialise all cells as UNKNOWN
   while goal not reached:
      observe adjacent cells
      update map and belief state
      if a safe known route exists:
         choose next step on lowest-cost route
      else:
         plan toward a safe unexplored frontier
      if next step is blocked or unsafe:
         update map and replan
      else:
         execute move and update cost
   report route and result
```

AI concepts and suitability

- Intelligent agent: senses and acts to achieve a rescue goal.
- Rationality: chooses the action expected to achieve the goal with low cost and acceptable risk using available information.
- Environment: partially observable, sequential, and potentially dynamic or stochastic.
- Online search is suitable because a complete map is unavailable initially; a purely offline method would require a known map.

Conclusion: online search combines sensing, belief updates, exploration, and risk-aware replanning.

4. MAP COLOURING OF SOUTH INDIAN STATES (CSP WITH HEURISTICS)

CSP and constraints

Variables: TN, KL, KA, AP, TG, GA, MH. Domain for each: {Red, Green, Blue}. Every listed neighbouring pair must have different colours. For this simplified example, constraints are TN–KL, TN–KA, TN–AP, KL–KA, KA–AP, KA–TG, KA–GA, KA–MH, AP–TG, TG–MH, GA–MH.

Heuristics and pseudocode

- MRV: choose the unassigned state with the fewest legal colours.
- Degree heuristic: break ties using the state with the most unassigned neighbours.
- Forward checking: remove the chosen colour from neighbours' domains; backtrack if a domain becomes empty.

```
MAP_COLOURING(assignment):
   if all states assigned: return assignment
   state = choose unassigned state using MRV
   for colour in {Red, Green, Blue}:
```

```
    if colour differs from assigned neighbours:
      assign state = colour
      forward-check neighbours
      if no domain is empty:
        result = MAP_COLOURING(assignment)
        if result succeeds: return result
      undo assignment and domain changes
  return FAILURE
```

State	Colour
Karnataka (KA)	Red
Tamil Nadu (TN)	Green
Kerala (KL)	Blue
Andhra Pradesh (AP)	Blue
Telangana (TG)	Green
Goa (GA)	Blue
Maharashtra (MH)	Green

This assignment satisfies the adjacency list specified above. A complete official map may include additional border constraints, which must also be checked.

5. 4 × 4 SUDOKU (CONSTRAINT PROPAGATION WITH AC-3)

Variables, domains, constraints

Variables are Cij for row i and column j, i,j∈{1,2,3,4}. Each empty cell initially has domain {1,2,3,4}; given cells have singleton domains. Cells in the same row, column, or 2 × 2 box must have different values.

Grid transcribed from the supplied image (underscore denotes a blank):

```
_ _ 2 _
3 _ _ 2
_ 3 _ 4
4 _ _ 1
```

AC-3 pseudocode

```
AC3(variables, domains, arcs):
  queue = all directed arcs (Xi, Xj)
  while queue is not empty:
    (Xi, Xj) = remove one arc
    if REVISE(Xi, Xj):
      if domain[Xi] is empty: return FAILURE
      add (Xk, Xi) for each other neighbour Xk
  return SUCCESS
```

```
REVISE(Xi, Xj):
    revised = false
    for each x in domain[Xi]:
        if no y in domain[Xj] satisfies x != y:
            remove x from domain[Xi]
            revised = true
    return revised
```

After propagation, use backtracking if any cell still has multiple candidates. Select the cell with the smallest remaining domain (MRV), try a value, run AC-3 again, and backtrack if a domain becomes empty. AC-3 prunes values that cannot be part of a consistent assignment.

6. CRYPTARITHMETIC PUZZLE (CSP WITH GLOBAL CONSTRAINTS)

Puzzle and variables

```
T W O
  + T W O
  -------
  F O U R
```

Letter variables: T, W, O, F, U, R, each representing a different digit 0–9. Carry variables C1, C2, C3 have domains {0,1}. Leading digits T and F cannot be zero.

Constraints and equations

```
2O = R + 10C1
2W + C1 = U + 10C2
2T + C2 = O + 10C3
C3 = F
AllDifferent(T, W, O, F, U, R)
T != 0, F != 0
```

Search pseudocode and solution

```
SOLVE_CRYPTO():
    assign letters using unused digits
    enforce leading-digit and AllDifferent constraints
    check each column equation when possible
    if all constraints hold and all variables assigned:
        return assignment
    if any constraint fails: backtrack
    continue trying digits; return FAILURE if none work
```

One valid solution is T=7, W=3, O=4, F=1, U=6, R=8: 734 + 734 = 1468. All digits are distinct, leading digits are non-zero, and the column equations hold.

7. AMBULANCE ROUTING DURING FLOODS (A* WITH ROAD-CLOSURE CONSTRAINT)

Road network and cost

Road	Cost	Road	Cost
A–B	3	C–D	2
S–B	2	C–E	3
A–C	4	D–G	2
B–C	1	E–G	4
B–D	Closed		

Heuristics supplied: h(S)=6, h(A)=6, h(B)=4, h(C)=2, h(D)=2, h(E)=3, h(G)=0. The prompt says route from A to B but also provides goal heuristic h(G)=0 and roads leading to G. For a coherent worked example using the full table, the following calculation treats A as start and G as destination; if the intended goal is B, the route is simply A→B with cost 3.

A* worked example

A* uses f(n)=g(n)+h(n). The closed road B–D is excluded. From A, B has g=3 and f=7; C has g=4 and f=6, so C is expanded first. From C, D has g=6 and f=8; E has g=7 and f=10. Expanding D reaches G at cost 8. Route A→C→D→G has total cost 4+2+2=8.

```
A_STAR_ROAD(start, goal):
   open = priority queue ordered by g+h
   best_g[start] = 0; insert start
   while open not empty:
      n = remove node with lowest f
      if n == goal: return path and best_g[n]
      for each road (n,m):
         if road is closed: continue
         new_g = best_g[n] + road_cost(n,m)
         if m is new OR new_g < best_g[m]:
            best_g[m] = new_g; parent[m] = n
            insert m with priority new_g+h(m)
   return FAILURE
```

A heuristic is admissible only if it never exceeds the true remaining minimum cost. The supplied values should be checked against the complete open-road graph; overestimation can remove A*'s optimality guarantee.

8. 4-QUEENS PROBLEM (LOCAL SEARCH WITH MIN-CONFLICTS)

CSP and constraints

Let Qi be the row of the queen in column i. Each variable Q1…Q4 has domain {1,2,3,4}. One queen per column prevents column conflicts. For $i \neq j$, require $Qi \neq Qj$ and $|Qi - Qj| \neq |i - j|$ to prevent same-row and diagonal attacks.

Min-conflicts pseudocode

```
MIN_CONFLICTS(max_steps):
   randomly assign one queen to each column
   for step = 1 to max_steps:
      if no queens conflict: return assignment
      choose a conflicted queen Qi
      move Qi to a row giving minimum conflicts
   return FAILURE
```

One valid solution is Q1=2, Q2=4, Q3=1, Q4=3:

```
. . . .
. . . Q
Q . . .
. . Q .
```

No queens share a row, column, or diagonal. Min-conflicts often works well for large CSPs because it changes only conflicted variables, but it may need restarts or a step limit.

9. MISSIONARIES AND CANNIBALS (STATE-SPACE SEARCH WITH SAFETY CONSTRAINTS)

State representation and constraints

Represent state as (M,C,B), where M and C are the numbers on the left bank and B is boat position (Left or Right). Initial state (3,3,Left); goal (0,0,Right). The boat carries one or two people and cannot cross empty.

Safety: on each bank, if missionaries are present, they must not be outnumbered by cannibals. Thus left bank: $M=0$ or $M \geq C$; right bank: $3-M=0$ or $3-M \geq 3-C$. Also $0 \leq M,C \leq 3$.

BFS pseudocode

```
BFS_MISSIONARIES():
   queue = [(3,3,LEFT)]
   visited = {(3,3,LEFT)}
   while queue not empty:
      state = remove_front(queue)
      if state == (0,0,RIGHT): return path_to(state)
      for passengers in {(2,0),(0,2),(1,1),(1,0),(0,1)}:
         next = cross(state, passengers)
         if within bounds and both banks safe and unseen:
            visited.add(next); parent[next] = state
```

```
            add next to queue
    return FAILURE
```

Minimum-crossing solution

Step	Move	Left-bank state (M,C)
Start	—	(3,3)
1	CC →	(3,1)
2	C ←	(3,2)
3	CC →	(3,0)
4	C ←	(3,1)
5	MM →	(1,1)
6	MC ←	(2,2)
7	MM →	(0,2)
8	C ←	(0,3)
9	CC →	(0,1)
10	C ←	(0,2)
11	CC →	(0,0)

BFS explores states in increasing order of crossings. Because each crossing has equal cost, the first goal state reached is a minimum-crossing solution. The standard solution takes 11 crossings.

10. SENSORLESS CORRIDOR ROBOT (SEARCH WITH PARTIAL INFORMATION)

Problem and belief state

A robot is in one of four corridor cells (1–4), but has no sensors. Initial belief state B0={1,2,3,4}. It can move Left or Right; attempting to move outside the corridor leaves it at the boundary. Goal: reach cell 4.

A belief state is the set of positions where the robot might be. With four positions, the set of possible belief states has at most 2^4=16 subsets.

Belief update and action sequence

For an action, apply the transition to every possible position and take the union of results. There is no observation to eliminate possibilities.

Action sequence	Belief state
Initial	{1,2,3,4}
Right	{2,3,4}
Right	{3,4}

Right	{4}

```
UPDATE_BELIEF(B, action):
    new_B = empty set
    for each position p in B:
        q = TRANSITION(p, action)
        add q to new_B
    return new_B
```

The open-loop plan [Right, Right, Right] guarantees that every possible initial position reaches cell 4. The robot cannot adapt to its exact position, so it plans over the belief state rather than a single known cell.